

Pointer

Programming for Problem Solving (PPS)

GTU # 3110003

USING

{C}

Programming



Prof. Nilesh Gambhava

Computer Engineering Department,
Darshan Institute of Engineering & Technology, Rajkot



What is Pointer?

- ▶ A normal variable is used to store value.
- ▶ A pointer is a variable that **store address / reference** of another variable.
- ▶ Pointer is **derived data type** in C language.
- ▶ A pointer contains the memory address of that variable as their value. Pointers are also called **address variables** because they contain the addresses of other variables.

Declaration & Initialization of Pointer

Syntax

```
1 datatype *ptr_variablename;
```

Output

```
10 10 5000
```

Example

```
1 void main()
2 {
3     int a=10, *p; // assign memory address of a
4     // to pointer variable p
5     p = &a;
6     printf("%d %d %d", a, *p, p);
7 }
```

Variable	Value	Address
a	10	5000
p	5000	5048



- ▶ **p** is integer pointer variable
- ▶ **&** is address of or referencing operator which returns memory address of variable.
- ▶ ***** is indirection or dereferencing operator which returns value stored at that memory address.
- ▶ **&** operator is the inverse of ***** operator
- ▶ **x = a** is same as **x = *(&a)**

Why use Pointer?

- ▶ C uses pointers to create **dynamic data structures**, data structures built up from blocks of memory allocated from the heap at run-time. Example linked list, tree, etc.
- ▶ C uses pointers to handle variable parameters passed to functions.
- ▶ Pointers in C provide an alternative way to **access information stored in arrays**.
- ▶ Pointer use in **system level programming** where memory addresses are useful. For example shared memory used by multiple threads.
- ▶ Pointers are used for file handling.
- ▶ This is the reason why C is versatile.

Pointer to Pointer – Double Pointer

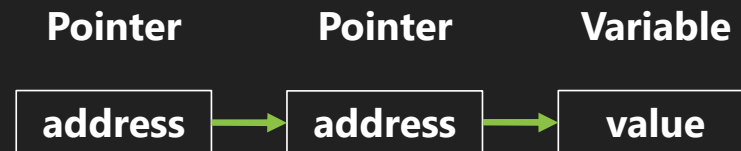
- ▶ Pointer holds the address of another variable of same type.
- ▶ When a pointer holds the **address of another pointer** then such type of pointer is known as **pointer-to-pointer or double pointer**.
- ▶ The first pointer contains the address of the second pointer, which points to the location that contains the actual value.

Syntax

```
1 datatype **ptr_variablename;
```

Example

```
1 int **ptr;
```



Write a program to print variable, address of pointer variable and pointer to pointer variable.

Program

```
1  #include <stdio.h>
2  int main () {
3      int var;
4      int *ptr;
5      int **pptr;
6      var = 3000;
7      ptr = &var; // address of var
8      pptr = &ptr; // address of ptr using address of operator &
9      printf("Value of var = %d\n", var );
10     printf("Value available at *ptr = %d\n", *ptr );
11         printf("Value available at **pptr = %d\n", **pptr);
12     return 0;
13 }
```

Output

```
Value of var = 3000
Value available at *ptr = 3000
Value available at **pptr = 3000
```

Relation between Array & Pointer

- ▶ When we declare an array, compiler allocates continuous blocks of memory so that all the elements of an array can be stored in that memory.
- ▶ The address of first allocated byte or the address of first element is assigned to an array name.
- ▶ Thus array name works as **pointer variable**.
- ▶ The address of first element is also known as **base address**.

Relation between Array & Pointer – Cont.

- ▶ Example: `int a[10], *p;`
- ▶ `a[0]` is same as `*(a+0)`, `a[2]` is same as `*(a+2)` and `a[i]` is same as `*(a+i)`

a:	a[0]
	a[1]
	.
	.
	.
	a[i]
	.
	.
	.
	a[9]

a:	*(a+0)	2000
a+1:	*(a+1)	2002
	.	
	.	
	.	
a+i:	*(a+i)	2000 + i*2
	.	
	.	
	.	
a+9:	*(a+9)	2018

Array of Pointer

- ▶ As we have an array of char, int, float etc, same way we can have an array of pointer.
- ▶ Individual elements of an array will store the address values.
- ▶ So, an array is a collection of values of similar type. It can also be a collection of references of similar type known by single name.

Syntax

```
1 datatype *name[size];
```

Example

```
1 int *ptr[5]; //declares an array of integer pointer of size 5
```

Array of Pointer – Cont.

- ▶ An array of pointers ptr can be used to point to different rows of matrix as follow:

Example

```
1 for(i=0; i<5; i++)  
2 {  
3     ptr[i]=&mat[i][0];  
4 }
```



- ▶ By dynamic memory allocation, we do not require to declare two-dimensional array, it can be created dynamically using array of pointers.

Write a program to swap value of two variables using pointer / call by reference.

Program

```
1  int main()
2  {
3      int num1,num2;
4      printf("Enter value of num1 and num2: ");
5      scanf("%d %d",&num1, &num2);
6
7      //displaying numbers before swapping
8      printf("Before Swapping: num1 is: %d, num2 is: %d\n",num1,num2);
9
10     //calling the user defined function swap()
11     swap(&num1,&num2);
12
13     //displaying numbers after swapping
14     printf("After Swapping: num1 is: %d, num2 is: %d\n",num1,num2);
15     return 0;
16 }
```

Output

```
Enter value of num1 and num2: 5
10
Before Swapping: num1 is: 5, num2 is: 10
After Swapping: num1 is: 10, num2 is: 5
```

Pointer and Function

- ▶ Like normal variable, pointer variable can be passed as function argument and function can return pointer as well.
- ▶ There are two approaches to passing argument to a function:
 - ➔ Call by value
 - ➔ Call by reference / address

Call by Value

- ▶ In this approach, the values are passed as function argument to the definition of function.

Program

```
1  #include<stdio.h>
2  void fun(int,int);
3  int main()
4  {
5      int A=10,B=20;
6      printf("\nValues before calling %d, %d",A,B);
7      fun(A,B);
8      printf("\nValues after calling %d, %d",A,B);
9      return 0;
10 }
11 void fun(int X,int Y)
12 {
13     X=11;
14     Y=22;
15 }
```

Output

Values before calling 10, 20
Values after calling 10, 20

Address	48252	24688		
Value	10	20	10 ¹¹	20 ²²
Variable	A	B	X	Y

Call by Reference / Address

- ▶ In this approach, the references / addresses are passed as function argument to the definition of function.

Program

```
1  #include<stdio.h>
2  void fun(int*,int*);
3  int main()
4  {
5      int A=10,B=20;
6      printf("\nValues before calling %d, %d",A,B);
7      fun(&A,&B);
8      printf("\nValues after calling %d, %d",A,B);
9      return 0;
10 }
11 void fun(int *X,int *Y)
12 {
13     *X=11;
14     *Y=22;
15 }
```

Output

Values before calling 10, 20
Values after calling 11, 22

Address	48252	24688		
Value	10 ¹¹	20 ²²	48252	24688
Variable	A	B	*X	*Y

Pointer to Function

- ▶ Every function has reference or address, and if we know the reference or address of function, we can access the function using its **reference or address**.
- ▶ This is the way of accessing function using pointer.

Syntax

```
1 return-type (*ptr-function)(argument list);
```

- ▶ **return-type**: Type of value function will return.
- ▶ **argument list**: Represents the type and number of value function will take, values are sent by the calling statement.
- ▶ **(*ptr-function)**: The parentheses around ***ptr-function** tells the compiler that it is pointer to function.
- ▶ If we write ***ptr-function** without parentheses then it tells the compiler that **ptr-function** is a function that will return a pointer.

Write a program to sum of two numbers using pointer to function.

Program

```
1  #include<stdio.h>
2  int Sum(int,int);
3  int (*ptr)(int,int);
4  int main()
5  {
6      int a,b,rt;
7      printf("\nEnter 1st number : ");
8      scanf("%d",&a);
9      printf("\nEnter 2nd number : ");
10     scanf("%d",&b);
11     ptr = Sum;
12     rt = (*ptr)(a,b);
13     printf("\nThe sum is : %d",rt);
14     return 0;
15 }
16 int Sum(int x,int y)
17 {
18     return x + y;
19 }
```

Output

Enter 1st number : 5

Enter 2nd number : 10

The sum is : 15

Practice Programs

1. Write a C program to print the address of variable using pointer.
2. Write a C a program to swap two elements using pointer.
3. Write a C a program to print value and address of a variable
4. Write a C a program to calculate sum of two numbers using pointer
5. Write a C a program to swap value of two numbers using pointer
6. Write a C a program to calculate sum of elements of an array using pointer
7. Write a C a program to swap value of two variables using function
8. Write a C a program to print the address of character and the character of string using pointer
9. Write a C a program for sorting using pointer



Thank you

